

Assignment 3

Shortest Path Algorithms: A Comparative Analysis

FAST NUCES ISB

Muhammad Abdullah Ali

Roll Num: 23i-2523

Section: 5A

Department of Data Science

Design and Analysis of Algorithms

November 23, 2025

1 OBJECTIVE AND APPROACH

1.1 Objective

The primary objective of this assignment is to implement, test, and analyze four classical shortest path algorithms: Dijkstra's Algorithm, Bellman-Ford Algorithm, Floyd-Warshall Algorithm, and Johnson's Algorithm. The study focuses on comparing algorithmic complexities and performance metrics (execution time, relaxations, and memory usage) under varying graph conditions, specifically Sparse ($E \approx V$), Dense ($E \approx V^2$), and Mixed graphs containing negative weights.

1.2 Approach

The algorithms were implemented in C++ to ensure high-performance execution. The graph representation was chosen based on the algorithm's requirements:

- **Adjacency List:** Utilized for Dijkstra, Bellman-Ford, and Johnson's algorithm to efficiently handle sparse graphs and minimize memory overhead.
- **Adjacency Matrix:** Utilized for the Floyd-Warshall algorithm to support the $O(1)$ edge lookups required by its dynamic programming structure.

Experiments were conducted on random connected graphs. Performance metrics were averaged over 500 runs to ensure statistical accuracy.

2 ALGORITHM PSEUDOCODE

2.1 Dijkstra's Algorithm

Dijkstra's algorithm solves the Single-Source Shortest Path (SSSP) problem for graphs with non-negative weights. It employs a greedy strategy, maintaining a priority queue of vertices ordered by their current shortest distance from the source. At each step, it extracts the vertex with the minimum distance and "relaxes" its neighbors—updating their distances if a shorter path is found. This greedy choice property ensures that once a node is finalized, its optimal distance is guaranteed.

Algorithm 1 Dijkstra(G, s)

```
1:  $dist[v] \leftarrow \infty, \forall v \in V; dist[s] \leftarrow 0$ 
2:  $Q \leftarrow \text{PriorityQueue}(V, \text{key}=dist)$ 
3: while  $Q \neq \emptyset$  do
4:    $u \leftarrow \text{ExtractMin}(Q)$ 
5:   for each  $v \in Adj[u]$  do
6:     if  $dist[u] + w(u, v) < dist[v]$  then
7:        $dist[v] \leftarrow dist[u] + w(u, v)$ 
8:        $\text{DecreaseKey}(Q, v, dist[v])$ 
9:     end if
10:  end for
11: end while
```

2.2 Bellman-Ford Algorithm

Unlike Dijkstra, the Bellman-Ford algorithm can handle graphs with negative edge weights. It works by iteratively relaxing all edges in the graph exactly $|V| - 1$ times. This is based on the path relaxation property: the shortest path between any two vertices can contain at most $|V| - 1$ edges. After these iterations, it performs one final pass to check for negative weight cycles. If any distance can still be improved, the graph contains a negative cycle, and no solution exists.

Algorithm 2 Bellman-Ford(G, s)

```
1:  $dist[v] \leftarrow \infty, \forall v \in V; dist[s] \leftarrow 0$ 
2: for  $i \leftarrow 1$  to  $|V| - 1$  do
3:   for each edge  $(u, v) \in E$  do
4:     if  $dist[u] + w(u, v) < dist[v]$  then
5:        $dist[v] \leftarrow dist[u] + w(u, v)$ 
6:     end if
7:   end for
8: end for
9: for each edge  $(u, v) \in E$  do
10:  if  $dist[u] + w(u, v) < dist[v]$  then return False (Negative Cycle)
11:  end if
12: end for
13: return True
```

2.3 Floyd-Warshall Algorithm

The Floyd-Warshall algorithm is a dynamic programming approach for the All-Pairs Shortest Path (APSP) problem. It constructs a solution by considering vertices $\{1, \dots, N\}$ as intermediate nodes. The recurrence relation $D_{ij} = \min(D_{ij}, D_{ik} + D_{kj})$ updates the shortest path between i and j by checking if a path through vertex k offers a shorter route. It runs in $O(V^3)$ and can handle negative weights, provided there are no negative cycles.

Algorithm 3 Floyd-Warshall(G)

```
1: Let  $D$  be the adjacency matrix
2: for  $k \leftarrow 1$  to  $N$  do
3:   for  $i \leftarrow 1$  to  $N$  do
4:     for  $j \leftarrow 1$  to  $N$  do
5:        $D[i][j] \leftarrow \min(D[i][j], D[i][k] + D[k][j])$ 
6:     end for
7:   end for
8: end for
```

2.4 Johnson's Algorithm

Johnson's algorithm is designed to efficiently solve the APSP problem on sparse graphs containing negative weights. It uses a technique called re-weighting. First, it runs Bellman-Ford (on a modified graph) to compute vertex potentials $h(v)$, which are used to transform edge weights $w(u, v)$ into non-negative values $\hat{w}(u, v) = w(u, v) + h(u) - h(v)$. Once weights are non-negative, it runs Dijkstra's algorithm for every vertex. This avoids the $O(V^3)$ complexity of Floyd-Warshall, achieving $O(VE + V^2 \log V)$.

Algorithm 4 Johnson(G)

```
1: Add vertex  $s'$  connected to all  $v$  with weight 0
2: if Bellman-Ford( $G, s'$ ) = False then return Error
3: end if
4: for  $v \in V$  do  $h[v] \leftarrow \delta(s', v)$ 
5: end for
6: for edge  $(u, v) \in E$  do  $\hat{w}(u, v) \leftarrow w(u, v) + h[u] - h[v]$ 
7: end for
8: for  $u \in V$  do
9:   Run Dijkstra( $G, \hat{w}, u$ ) to find  $\hat{\delta}(u, v)$ 
10:   $d(u, v) \leftarrow \hat{\delta}(u, v) + h[v] - h[u]$ 
11: end for
```

3 SAMPLE INPUT/OUTPUT

The implementation was verified using standard test cases. Below represents a sample execution showing the menu-driven interface and path calculation.

```
1. Solve Input from .txt file
2. Run Experimental Analysis
Enter choice: 1

Found existing 'input.txt'. Using it.
-> Press ENTER once 'input.txt' is ready...
Is the graph Directed? (1 for Yes, 0 for No): 1

=====
LOADED GRAPH STRUCTURE (V=5)
=====
Vertex 1 connects to:
-> Vertex 2 (weight: 2)
-> Vertex 3 (weight: 4)
Vertex 2 connects to:
-> Vertex 3 (weight: 1)
-> Vertex 4 (weight: 7)
Vertex 3 connects to:
-> Vertex 5 (weight: 3)
Vertex 4 connects to:
-> Vertex 5 (weight: 1)
Vertex 5 connects to:
-> Vertex 4 (weight: 2)
=====

Processing Algorithms...

Algorithm: Dijkstra (Src: 1)
-----
Vertex  Distance
1          0
2          2
3          3
4          8
5          6
-----

Algorithm: Bellman-Ford (Src: 1)
-----
Vertex  Distance
1          0
2          2
3          3
4          8
5          6
-----

Algorithm: Floyd-Warshall
-----
0  2  3  8  6
INF 0  1  6  4
INF INF 0  5  3
INF INF INF 0  1
INF INF INF 2  0
-----

Algorithm: Johnson
-----
0  2  3  8  6
INF 0  1  6  4
INF INF 0  5  3
INF INF INF 0  1
INF INF INF 2  0
PS D:\work\Semester5\AlgoAssignments\Asst-3> []
```

Figure 1: Screenshot of the program execution and result verification.

4 EXPERIMENTAL ANALYSIS

The algorithms were tested on three distinct graph types. The image below represents the metrics for one run.

```

1. Solve Input from .txt file
2. Run Experimental Analysis
Enter choice: 2
Running Experimental Analysis...
-> Saved visualization to 'viz_Sparse.dot'

=====
ANALYSIS: Sparse Graph (V=15, E=16)
=====
Algorithm   Time(ms)  Relaxations  Mem(KB)  Neg Weights?  Complexity
-----
Dijkstra    0.004     4            0.72     No             O((V+E)logV)
Bellman-Ford 0.004     64           0.72     Yes            O(VE)
Floyd-Warsh 0.035     3375         3.52     Yes            O(V^3)
Johnson    0.074     572          1.09     Yes            O(V^2logV + VE)
-----
-> Saved visualization to 'viz_Dense.dot'

=====
ANALYSIS: Dense Graph (V=157, E=19593)
=====
Algorithm   Time(ms)  Relaxations  Mem(KB)  Neg Weights?  Complexity
-----
Dijkstra    0.465     19593        311.05   No             O((V+E)logV)
Bellman-Ford 0.979     78372        311.05   Yes            O(VE)
Floyd-Warsh 48.145    3869893      385.14   Yes            O(V^3)
Johnson    95.746    6196601      618.41   Yes            O(V^2logV + VE)
-----
-> Saved visualization to 'viz_Mixed.dot'

=====
ANALYSIS: Mixed Graph (V=41, E=82)
=====
Algorithm   Time(ms)  Relaxations  Mem(KB)  Neg Weights?  Complexity
-----
Dijkstra    0.000     0            0.00     No (Skipped)   O((V+E)logV)
Bellman-Ford 0.100     3280         2.56     Yes            O(VE)
Floyd-Warsh 1.904     68921        26.27    Yes            O(V^3)
Johnson    0.148     5166         4.16     Yes            O(V^2logV + VE)
-----

(Tip: Copy text from .dot files to https://dreampuf.github.io/GraphvizOnline/ to view graphs)
PS D:\Work\Semester5\Algo\Assignments\Asst-3>

```

Figure 2: Visual representation of the Graph structures used.

4.1 Performance Comparison

Table 1: Comparative Analysis Results (Average of 500 Runs)

Algorithm	Time (ms)	Relaxations	Memory (KB)	Complexity
<i>Sparse Graphs ($E \approx V$)</i>				
Dijkstra	0.0018	6.1	0.54	$O(E + V \log V)$
Bellman-Ford	0.0009	17.2	0.54	$O(VE)$
Floyd-Warshall	0.0215	371.6	0.94	$O(V^3)$
Johnson	0.0579	292.8	0.63	$O(VE + V^2 \log V)$
<i>Dense Graphs ($E \approx V^2$)</i>				
Dijkstra	0.3718	18,493	207.1	$O(E + V \log V)$
Bellman-Ford	0.3870	56,514	207.1	$O(VE)$
Floyd-Warshall	19.565	3.7M	128.0	$O(V^3)$
Johnson	58.427	3.0M	208.1	$O(VE + V^2 \log V)$
<i>Mixed Graphs (Negative Weights)</i>				
Dijkstra	N/A	N/A	N/A	Fails
Bellman-Ford	0.0063	413.6	2.56	$O(VE)$
Floyd-Warshall	0.0680	4,101	13.1	$O(V^3)$
Johnson	0.1998	2,023	2.88	$O(VE + V^2 \log V)$

4.2 Discussion of Results

- **Sparse Graphs:** Bellman-Ford (0.0009 ms) surprisingly outperformed Dijkstra (0.0018 ms) in this specific micro-benchmark. This is attributed to the low number of vertices ($V = 10 - 50$), where the overhead of maintaining a Priority Queue in Dijkstra exceeds the simple iterative approach of Bellman-Ford.
- **Dense Graphs:** Floyd-Warshall (19.5 ms) performed significantly better than Johnson's algorithm (58.4 ms). Johnson's algorithm runs Dijkstra $|V|$ times; in dense graphs ($E \approx V^2$), the complexity rises, making it slower than the tight loops of Floyd-Warshall's $O(V^3)$.
- **Mixed Graphs:** Dijkstra was unable to run due to negative weights. Floyd-Warshall provided the fastest All-Pairs solution for these small graphs.

5 REPORT QUESTIONS

5.1 Differences between Single-Source and All-Pairs Algorithms

Single-Source Shortest Path (SSSP) algorithms, such as Dijkstra and Bellman-Ford, calculate the shortest path from one specific source node to all other nodes in the graph. In contrast, All-Pairs Shortest Path (APSP) algorithms, such as Floyd-Warshall and Johnson's, compute the shortest paths between every possible pair of vertices (u, v) in the graph.

5.2 Why Dijkstra's Algorithm Fails with Negative Weights

Dijkstra's algorithm relies on a "greedy" strategy. It assumes that once a vertex is extracted from the priority queue (visited), its optimal distance is finalized and will never decrease. If negative edges exist, a path discovered later through a "more expensive" initial node might actually lead to a shorter total distance by traversing a negative edge. Dijkstra's greedy assumption blinds it to this possibility.

5.3 How Johnson's Algorithm Improves Efficiency

Johnson's algorithm combines the strengths of Bellman-Ford and Dijkstra. It first runs Bellman-Ford once to compute a "re-weighting" function that transforms all edge weights to non-negative values while preserving the shortest path structure. Once weights are non-negative, it runs Dijkstra's algorithm $|V|$ times. This allows it to handle negative weights while enjoying the efficiency of Dijkstra for sparse graphs.

5.4 Complexity Analysis

This section details the derivation of time complexity for each algorithm.

Dijkstra's Algorithm: The algorithm performs two main operations:

- **Extract-Min:** Executed once for each vertex $|V|$. Using a Binary Heap, this takes $O(\log V)$, totaling $O(V \log V)$.

- **Decrease-Key:** Executed for each edge $|E|$ during relaxation. In a Binary Heap, this takes $O(\log V)$, totaling $O(E \log V)$.

Combining these, the total complexity is $O((V + E) \log V)$.

Bellman-Ford Algorithm: The algorithm works by strictly relaxing all edges $|E|$ in the graph exactly $|V| - 1$ times.

- In each pass (1 to $|V| - 1$), the algorithm iterates over all $|E|$ edges.
- Therefore, the total work is simply the product of passes and edges: $O(V \times E)$.

Floyd-Warshall Algorithm: The algorithm uses a dynamic programming approach with three nested loops:

- **Outer Loop (k):** Iterates through each vertex as an intermediate node (1 to $|V|$).
- **Middle Loop (i):** Iterates through each source node (1 to $|V|$).
- **Inner Loop (j):** Iterates through each destination node (1 to $|V|$).

Since each operation inside the inner loop is constant time, the total complexity is $|V| \times |V| \times |V| = O(V^3)$.

Johnson's Algorithm: Johnson's algorithm consists of two distinct phases:

- **Phase 1 (Reweighting):** Runs Bellman-Ford once to eliminate negative weights. Cost: $O(VE)$.
- **Phase 2 (All-Pairs):** Runs Dijkstra's algorithm once for every vertex in the graph ($|V|$ times).
- Since one run of Dijkstra is $O(V \log V + E)$ (using an optimal heap), running it $|V|$ times results in $O(V^2 \log V + VE)$.

The total complexity is the sum of both phases: $O(V^2 \log V + VE)$.

5.5 Algorithm Suitability

Based on the experimental data:

1. **Sparse Graphs (Non-negative):** Dijkstra is theoretically optimal, although Bellman-Ford remains efficient for very small graphs.
2. **Dense Graphs (APSP):** Floyd-Warshall is the superior choice due to lower constant factors and simpler implementation compared to Johnson's.
3. **Sparse Graphs (Negative Weights):** Johnson's algorithm is the most suitable for APSP, while Bellman-Ford is required for SSSP.

6 CONCLUSION

This assignment highlighted the trade-offs between shortest path algorithms. While Floyd-Warshall is elegant and efficient for dense graphs, it lacks scalability. Johnson's algorithm bridges the gap for sparse graphs containing negative weights. For single-source problems, Dijkstra remains the gold standard for non-negative weights, while Bellman-Ford is indispensable for graphs with negative edges, despite its higher computational cost.